Programming Project: Morris AI Tournament

CS4346: Intro to Artificial Intelligence

Team size: 3 students
Tournament scheduling deadline: 10/08/2026
Tournament completion deadline: 10/15/2026
Final project submission deadline: 10/08/2026

Overview

In this project you will develop an Artificial Intelligence agent for the **Morris Game, Variant** described in the accompanying handout.

The project begins with implementations of MINIMAX and ALPHA-BETA search. You will then improve the agent by developing your own static estimation function.

The final part of the project is a **class tournament**. Your team will play its agent against agents developed by other teams.

The purpose of the tournament is not only to determine which agent wins. It is intended to demonstrate an important distinction:

A program can implement an AI algorithm correctly without necessarily being a strong game-playing agent.

The strength of your tournament agent will depend on several factors, including:

- correctness of the move generator,
- correctness of MINIMAX and ALPHA-BETA,
- quality of the static estimation function,
- search depth,
- computational efficiency,
- and the ability to make decisions within the tournament time limit.

Important

Tournament ranking itself does **not** determine your project grade. A team is not penalized simply because another team's agent is stronger.
The tournament portion is graded primarily on correctness, participation, completion of assigned games, and compliance with the tournament protocol.

1. Game Representation

A board position is represented using a string of length 23.

Each character is one of:

- `W` – a White piece,
- `B` – a Black piece,
- `x` – an empty location.

For example,

`xxxxxBxWWWWWBBBBxxxxxxx`

represents one possible board position.

The mapping between the 23 characters and the physical board locations is given in the *Morris Game, Variant* handout.

The game has three phases:

1. **Opening:** each player places nine pieces.
2. **Midgame:** pieces move to adjacent empty locations.
3. **Endgame:** a player with only three pieces may hop to any empty location.

When a player closes a mill, one eligible opponent piece is removed according to the rules described in the Morris handout.

2. Part I: MINIMAX (25%)

Write programs implementing MINIMAX using the algorithms discussed in class.

Your programs should use the move generators and static estimation functions described in the Morris handout.

2.1. MiniMaxOpening

Write a program called

`MiniMaxOpening.py`

which computes White's best move during the opening phase.

The program receives:

1. an input board file,
2. an output board file,
3. and a search depth.

For example:

Example

```
python MiniMaxOpening.py board1.txt board2.txt 2
```

The program should print:

```
Board Position: ...
Positions evaluated by static estimation: ...
MINIMAX estimate: ...
```

The selected board position should also be written to the output file.

2.2. MiniMaxGame

Write a second program called

`MiniMaxGame.py`

which computes White's best move during the midgame/endgame phase.
Its input and output format should be identical to `MiniMaxOpening.py`.

3. Part II: ALPHA-BETA (20%)

Rewrite the programs from Part I using ALPHA-BETA pruning.
The programs should be called:

`ABOpening.py`
`ABGame.py`

For the same position and search depth, MINIMAX and ALPHA-BETA should produce the same MINIMAX estimate and the same optimal move, assuming the same tie-breaking behavior.
The primary difference should be the number of positions evaluated.

Note

ALPHA-BETA pruning does not change the result of MINIMAX. It avoids evaluating parts of the search tree that cannot affect the final decision.

Your examples should contain at least two cases in which ALPHA-BETA evaluates fewer positions than MINIMAX.

4. Part III: Playing for Black (10%)

Your programs so far compute moves for White.
Modify your implementation so that your agent can also compute moves for Black.
You should produce:

`MiniMaxOpeningBlack.py`
`MiniMaxGameBlack.py`

You may use the color-swapping method discussed in the Morris handout or another correct implementation consistent with the algorithms presented in class.
Being able to play both White and Black will be required for the tournament.

5. Part IV: Improved Static Estimation (20%)

The static estimation functions given in the handout are deliberately simple.
Design a new static estimation function that you believe will produce a stronger Morris player.
Your evaluation may consider features such as:

- difference in number of pieces,
- number of possible moves,

- mobility,
- completed mills,
- potential future mills,
- blocked opponent pieces,
- strategically useful board locations,
- or other features that you believe are useful.

You are not required to use all of these features.
Your function should be based on a clear game-playing rationale.
Produce:

`MiniMaxOpeningImproved.py`
`MiniMaxGameImproved.py`

Also submit:

`MyStaticEstimation.txt`

The file should explain:

1. the features used by your evaluation function,
2. why you selected them,
3. how they are combined,
4. and why you believe the resulting evaluation is better than the original evaluation function.

Provide at least two example board positions where your improved evaluation function selects a different move from the original evaluation function.
Briefly explain why you prefer the move selected by your new function.

6. Part V: Tournament Agent (25%)

The final part of the project is to use the components developed above to create the strongest Morris-playing agent that your team can produce.
Your tournament agent must:

- make legal moves,
- play both White and Black,
- correctly handle the opening, midgame, and endgame,
- use MINIMAX or ALPHA-BETA as described in class,
- and respond within the tournament move time limit.

You may choose the search depth used by your agent.
You may also use your improved static estimation function.

Important

Your agent must follow the search algorithms discussed in class.
The tournament is intended to compare implementations and evaluation strategies based on those algorithms, not unrelated game-playing algorithms.

You may combine the components developed in Parts I–IV into one tournament program if you wish.

If you create a separate tournament program, name it:

`TournamentAgent.py`

7. Tournament Format

Each team will be assigned at least **three other teams** as official opponents.

Because the number of teams in the class may not allow a perfectly symmetric schedule, some teams may be assigned one additional official opponent.

The instructor will release the official matchup list.

7.1. Two Games per Matchup

Each official matchup consists of **two games**.

If Team A plays Team B:

- **Game 1:** Team A plays White and Team B plays Black.
- **Game 2:** Team B plays White and Team A plays Black.

Therefore, both agents are tested from both sides of the game.

7.2. Scheduling Matches

After tournament matchups are released, teams must contact their assigned opponents and agree on a time to play.

All official games must be scheduled by:

10/08/2026

All official tournament games must be completed by:

10/15/2026

If two teams are unable to agree on a time before the scheduling deadline, they must contact the instructor.

The instructor may assign a match time.

8. Playing an Official Match

For each matchup, the two teams should create a Microsoft Teams group chat containing the members of both teams.

For teams of three students, this will normally be a six-person chat.

The Teams conversation will serve as the official record of the game.

8.1. Designated Communicator

For each game, each team should designate one member responsible for posting the team's moves.

Other team members may discuss the move and help operate the program, but only the designated communicator should post the official board position.

This prevents multiple conflicting moves from being submitted.

8.2. Starting the Game

The teams should begin at the agreed match time.

White makes the first move.

The first official board position posted by White begins the game.

For each move, the team should post:

1. the move number,
2. the side that made the move,
3. and the 23-character board position.

For example:

Example

```
Move 7 - White
xxxWxxxxBxxWWxxxxBxxxxx
```

Teams should keep track of the move number throughout the game.

The first 18 placement moves constitute the opening phase: each player places nine pieces.

After both players have placed all nine pieces, play proceeds according to the midgame/endgame rules.

9. Move Time Limit

After a legal board position is posted, the opposing team has

4 minutes

to post its response.

The Teams timestamp will be used to determine whether the move was submitted within the allowed time.

The purpose of the time limit is to make computational efficiency part of the tournament.

A team may therefore need to balance:

- search depth,
- quality of decision making,
- and computation time.

9.1. Timeout

If a team fails to post a move within 4 minutes, the opposing team may claim the game as a win.
However, the opposing team may voluntarily allow the game to be restarted.
This is entirely at the discretion of the opposing team.
A team is never required to waive a timeout.

Example

Suppose Team A posts a position at 7:10 PM.
Team B must post its response no later than 7:14 PM.
If Team B responds at 7:15 PM, Team A may claim the game.
Team A may instead choose to restart the game if it wishes.

10. Illegal Moves

Every submitted position must represent a legal move from the previous board position.
If a team submits an illegal position, the opposing team may claim the game as a win.
However, an incorrect board position may be corrected if:

1. the correction is made before the original 4 minutes expires,
2. and the opposing team has not yet responded to the incorrect position.

Once the opponent has responded, the previous move is considered final.
If there is disagreement about whether a move is legal, save the Teams conversation and contact the instructor.

11. Winning the Game

A player wins according to the rules of the Morris Game, Variant.
A player wins if the opponent:

- is reduced to two pieces, or
- has no legal move available.

A game may also be won by:

- opponent timeout,
- or an uncorrected illegal move.

12. Draws

To prevent indefinitely repeating games, a game is declared a draw if the **same board position with the same player to move occurs three times**.

Both teams should identify the repeated position in the Teams conversation.

If the teams disagree about whether a repetition has occurred, the game log should be sent to the instructor.

13. Tournament Standings

Each individual game is scored as follows:

Result	Tournament Points
Win	1
Draw	0.5
Loss	0

Because some teams may play a different number of official games, standings will be based primarily on:

$$\text{Tournament Score} = \frac{\text{Points Earned}}{\text{Number of Games Played}}$$

rather than simply the total number of wins.

Tournament standings are intended primarily for the class competition and do not directly determine the project grade.

14. Additional Practice Matches

Teams are welcome to play additional games against other teams after completing their required official matchups.

These additional games are considered **practice matches only**.

They:

- do not count toward the official tournament standings,
- should not be submitted through the tournament website,
- and will not be displayed on the tournament website.

Only instructor-assigned matchups are considered official tournament games.

Teams are encouraged to use practice matches to test improvements to their agents, experiment with different search depths, and evaluate changes to their static estimation functions.

15. Reporting Match Results

After both games in an official matchup have been completed, one of the two teams must record the result in the shared tournament results document (link on Canvas).

Each team will be assigned a unique tournament identifier such as

`T01`, `T02`, ...

These identifiers must be used when reporting results.

15.1. Result Format

Each completed matchup must be entered on **one line** using exactly the following format:

```
MATCH|TeamA|TeamB|Game1|Game2|YYYY-MM-DD
```

The following result codes must be used:

Code	Meaning
`A`	Team A won the game
`B`	Team B won the game
`D`	The game was a draw

Recall that the colors are determined by the order of the teams:

- **Game 1:** Team A plays White and Team B plays Black.
- **Game 2:** Team B plays White and Team A plays Black.

15.2. Example

Suppose Team `T01` and Team `T07` play their official matchup.
Team `T01` wins Game 1, and Team `T07` wins Game 2.
The result should be entered as:

```
MATCH|T01|T07|A|B|2026-10-15
```

As another example,

```
MATCH|T03|T09|A|D|2026-10-17
```

means that Team `T03` won Game 1 and Game 2 ended in a draw.

Important

Do not change the formatting, add spaces between fields, or use team names instead of the assigned team identifiers.
Only instructor-assigned official matchups should be entered in the results document. Practice matches should not be reported.

The Teams conversation remains the official record of the games if a reported result is disputed.

16. Program Development

Your team is responsible for designing, implementing, testing, and debugging the programs required for this project.

You may use any programming and development tools or resources that are permitted under course and university policies. The specific development workflow you use is up to your team.

Regardless of the tools used, your team is responsible for:

- ensuring that the submitted programs correctly implement the algorithms described in class,

- testing the programs on a variety of board positions,
- understanding how your implementation works,
- identifying and correcting errors,
- and being able to explain the major design decisions in your implementation.

Important

The use of development tools does not change your responsibility for the submitted work. Your team should understand the code you submit and should be able to explain how the MINIMAX, ALPHA-BETA, move generation, and static estimation components of your program work.

17. Team Collaboration

This is a team project.

Members of the same team may:

- discuss the algorithms,
- jointly develop prompts,
- test the programs,
- debug the implementation,
- design the static estimation function,
- and operate the tournament agent.

Teams may discuss general ideas and tournament performance with other teams. However, different teams may not share:

- Python source code,
- complete prompts,
- implementation-specific solutions,
- or their complete static estimation implementation.

18. What to Submit

Submit one ZIP file per team.

The ZIP file should contain:

1. `MiniMaxOpening.py`
2. `MiniMaxGame.py`
3. `ABOpening.py`

4. `ABGame.py`
5. `MiniMaxOpeningBlack.py`
6. `MiniMaxGameBlack.py`
7. `MiniMaxOpeningImproved.py`
8. `MiniMaxGameImproved.py`
9. `Examples.txt`
10. `MyStaticEstimation.txt`
11. `TournamentAgent.py`, if your team created a separate tournament program.

19. Grading

The project will be graded approximately as follows:

Component	Weight
Part I: MINIMAX	25%
Part II: ALPHA-BETA	20%
Part III: Playing for Black	10%
Part IV: Improved Static Estimation	20%
Part V: Tournament Agent and Participation	25%
Total	**100%**

For the tournament component, grading will consider:

- whether the agent produces legal moves,
- whether the team can play both colors,
- completion of assigned tournament games,
- compliance with the time limit and tournament protocol,
- and appropriate participation by the team.

Tournament wins and losses themselves are not the primary basis for the project grade.

20. Team Grades

This is a team project. In general, all members of a team will receive the same grade for the project.

The team will submit one project submission, and the implementation, documentation, and tournament participation will be evaluated as a single team effort.

Important

The shared team grade assumes that all members contribute reasonably to the project. If there is a significant participation issue within a team, students should inform the instructor as early as possible. In exceptional cases, individual grades may be adjusted based on documented differences in participation.